

Services

Tim Schmitz
05.07.2023

Inhalt

Definition	3
Registrierung	3

Services

Wie jeder andere Baustein auch ist ein Service eine Klasse in TypeScript. Ein Service definiert die Logik eines Anwendungsteils und enthält auch noch Hilfsmethoden. Diese Klasse wird aber erst dann zu einem Service, wenn sie von einem Provider verwendet wird. Alle Bausteine, die von diesem Service abhängen, erhalten diesen dann auch mittels Dependency Injection.

Definition

Das folgende Beispiel ist ein schlichter Service, der als einzige Funktion einen Array von Zahlen zurückgibt:

```
import { Injectable } from '@angular/core';

@Injectable()
export class DemoService {
  getData() {
    return [1, 2, 3];
  }
}
```

Diese Klasse wird mit dem Injectable-Decorator dekoriert. Somit erhält auch diese Klasse wie auch die Klassen von Komponenten und Angular-Modulen Metadaten. Der Service kann ohne weiteres nicht genutzt werden. Sondern er muss als erstes registriert werden.

Wenn man eine neue Klasse (hier die Klasse DemoService als Datei demo.service.ts im Verzeichnis demo) einem Projekt hinzufügt, dann folgt manchmal die Warnung (im Visual Studio Code): "To enable code execution from the current workspace you must enable workspace library execution". Dies erfolgt wie folgt:

1. Aufrufen der Kommandopalette mit Ctrl + Shift + P
2. Eingeben von "TSLint: Manage workspace library execution"
3. Aufnehmen des entsprechenden Eintrags aus der Vorschlagsliste und die Return-Taste bedienen

Registrierung

Bevor die Klasse DemoService zu einem Service wird und als solcher genutzt werden kann, muss er zunächst als Provider registriert werden. Um dies zu erreichen, gibt es zwei Möglichkeiten:

1. Die Service-Klasse kann von dem Angular-Module in einem providers-Array registriert werden
2. Die Service-Klasse kann sich über den Injectable-Decorator selbst registrieren

In der ersten Variante das Angular-Module AppModule den providers-Array nutzen, um die Service-Klasse zu registrieren:

```
import { DemoService } from '../demo/demo.service';
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
import { AppComponent } from './app.component';

@NgModule({
  declarations: [ AppComponent ],
  imports: [ BrowserModule ],
  providers: [ DemoService ],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Da der DemoService im Verzeichnis demo neben dem Hauptverzeichnis app liegt, muss die Import-Direktive den relativen Pfad qualifizieren. Die zweite Variante nutzt für die Selbstregistrierung wieder den Injectable-Decorator der Klasse DemoService.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class DemoService {
  getData() {
    return [1, 2, 3];
  }
}
```

Es wird ein Objekt mit der Eigenschaft "providedIn" übergeben. Der Wert dieser Eigenschaft gibt den Injektor an, in dem der Service als Provider übergeben werden soll. Die Wertausprägung "root" legt nun fest, dass der Service im globalen Injektor registriert werden soll. Der globale Injektor ist dabei identisch mit dem Angular-Hauptmodule AppModule. Man könnte auch ein Angular-Modul oder eine Komponente als Injektor angeben. Diese Providers heißen tree-shakeable Providers und sind bevorzugt im Vergleich zu der Registrierung über providers-Arrays zu verwenden, da sie nur im JavaScript Code-Bundle verbleiben, wenn sie auch benutzt werden.